

DEPLOYMENT PLAN

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control	2
FINAL ROUND (Technical Architecture & Deployment Plan)	3
1. Introduction	3
2. Technical Stack & Justification	3
3. CI/CD Pipeline & Automation	6
4. Secure Integration & Credential Management	8

1. Introduction

This document outlines the technical architecture, deployment strategy, CI/CD pipeline, and credential management approach for Homie — an AI-powered rental search aggregator designed for the Malaysian residential rental market. It is intended to justify each major technology choice, explain how the system scales under load, and demonstrate that sensitive credentials and third-party integrations are managed securely.

The document covers three interconnected pillars:

- Justified Technical Stack & Scalability — why each component was chosen and how the system grows.
- CI/CD Pipeline & Automation — how code moves from development to production reliably.
- Secure Integration & Credential Management — how API keys, session tokens, and secrets are protected.

2. Technical Stack & Justification

2.1 System Overview

Homie follows a client-server architecture with an asynchronous Python backend, a React frontend, a PostgreSQL data store, and ILMU-GLM-5.1 as its AI orchestration layer. All components are stateless or session-scoped, enabling horizontal scaling without a shared mutable state.

2.2 Backend — Python + FastAPI

FastAPI was selected as the REST framework for the following reasons:

- Native async/await support allows concurrent scraping of four listing sources without blocking the event loop.
- Pydantic models enforce strict input validation at the API boundary, eliminating an entire class of runtime errors.
- Automatic OpenAPI schema generation reduces integration friction with the frontend team.
- Uvicorn (ASGI) achieves significantly higher throughput than WSGI alternatives (Flask, Django) for I/O-bound workloads.

Component	Choice	Justification
REST Framework	FastAPI	Async-first, Pydantic validation, auto OpenAPI
ASGI Server	Uvicorn	High-throughput async; production-grade

© UMHackathon 2026

Email: umhackathon@um.edu.my

ORM	SQLAlchemy 2.x	Async sessions, schema migrations, PostgreSQL + SQLite parity
HTTP Scraping	httpx + BS4	Async HTTP client; lightweight for iBilik REST API
Browser Auto.	Playwright	Headless Chromium; handles JS-heavy sites & Cloudflare
Telegram Client	Telethon (MTPROTO)	MTPROTO API; full conversation management & message sending
Package Mgr	uv	10× faster than pip; reproducible lockfiles
Process Mgr	PM2	Autorestart, log aggregation, zero-downtime reload

2.3 Frontend — Next.js 14

Next.js 14 (React Server Components + App Router) was chosen over a pure SPA because:

- Server-side rendering improves initial load time and perceived performance on low-bandwidth connections.
- The App Router enables co-located data fetching per route segment, reducing client-side waterfall requests.
- Native EventSource support handles the SSE progress stream without additional libraries.
- Tailwind CSS v4 + shadcn/ui provides a consistent, accessible component set with minimal bundle overhead.

2.4 Database — PostgreSQL (Neon)

PostgreSQL was chosen over NoSQL alternatives because the listing schema is relational (Sessions → Listings → OutreachEvents) and benefits from foreign-key integrity and transactional writes during the pipeline. Neon (serverless PostgreSQL) was selected for:

- Instant branching for staging environments without provisioning overhead.
- Autoscaling compute-on-demand, billing only for active connection time.
- Built-in connection pooling via PgBouncer, essential for Uvicorn's many short-lived async connections.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- SQLite is used locally and in the CI test harness to avoid network dependencies in the test environment.

2.5 AI Orchestration — ILMU-GLM-5.1 (Z.AI via YTL AI Labs)

ILMU-GLM-5.1 is integrated as a reasoning engine through an OpenAI-compatible API proxy hosted by YTL AI Labs. The choice is justified by:

- Native structured tool/function calling, essential for the ReAct orchestrator's multi-step tool invocation.
- Strong multilingual instruction-following (Bahasa Malaysia, Chinese, English) without fine-tuning.
- OpenAI-compatible API, allows a drop-in model swap if the provider changes.

Numeric scoring is intentionally implemented as deterministic backend code, not model-generated, to eliminate score variance across runs and reduce token consumption. The model is only invoked for reasoning-heavy tasks: filter extraction, normalization, explanation generation, report summarization, and outreach drafting.

2.6 Scalability Strategy

The current deployment is scoped for hackathon-scale use (single concurrent user). The architecture is intentionally designed to scale with minimal rework:

Dimension	Current (Hackathon)	Production Path
Concurrency	Single user, in-process session store	Migrate session state to Redis; use async worker pool
Scraping load	Playwright in-process, 4 sources × 25 listings	Move to dedicated scraper workers via Celery + RabbitMQ
DB connections	SQLAlchemy async pool, 5 connections	Neon PgBouncer + connection pool scaling
API layer	Single Uvicorn instance via PM2	Multiple Uvicorn workers; Nginx upstream or Cloud Run
AI token budget	~8,000 tokens/session, capped	Prompt caching layer for repeated normalization calls
Deduplication	URL-based only	Vector embedding similarity for cross-source dedup
Frontend	Next.js on port 3000 via PM2	Vercel or Cloud Run with edge CDN

The NFR targets from the SAD remain the scalability benchmarks: P95 latency ≤ 8s, error rate < 1%, scraper success rate > 95% per source, and average token consumption below 10,000 per sessi

3. CI/CD Pipeline & Automation

3.1 Pipeline Overview

Homie follows a trunk-based development model with a single long-lived main branch that must always be deployable. Feature work is developed on short-lived branches and merged via pull request. The CI/CD pipeline enforces quality gates at two checkpoints: integration (merge to main) and deployment (push to production).

3.2 Integration Gates (Merge to Main)

The following checks must all pass before a branch may be merged:

Gate	Tool / Command	Pass Condition
Backend smoke import	uv run python -c 'import main'	Zero import/runtime init errors
Automated backend tests	uv run pytest -q	≥1 executable suite; 0 failures
Critical flow smoke	FastAPI TestClient + DEMO_SEED=true	Pipeline completes; ≥1 listing; draft succeeds
Frontend production build	cd frontend && npm run build	Exit 0; no blocking dependency errors
AI fallback verification	Failure-injection harness	Pipeline completes with fallback explanation/report
Type checking (frontend)	npx tsc --noEmit	Zero type errors

3.3 Deployment Gates (Push to Production)

Gate	Requirement	Current Result
Regression core flow	100% of critical smoke scenarios pass	4/4 passed in controlled harness
Frontend release build	Successful production build	Passed
AI output contract	Outputs meet acceptance criteria or safe fallback	Passed — failure injection verified
API readiness	GET /results P95 < 10ms locally	Avg 1.66ms, max 2.11ms over 20 samples
DB migration check	Schema matches SQLAlchemy models	Verified via alembic check or model diff

3.4 Automation Workflow

The recommended CI automation sequence per pull request is:

Trigger: Push to any branch or open PR targeting main

Env setup: uv sync --frozen, reproducible dependency installation from lockfile

Backend smoke: uv run python -c 'from main import app', fast import check

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test suite: `uv run pytest -q --tb=short, unit + integration with temp SQLite`

Fallback inject: `uv run pytest tests/test_fallback.py`, AI failure injection suite

Frontend build: `cd frontend && npm ci && npm run build`

Type check: `cd frontend && npx tsc --noEmit`

Merge gate: All steps green → PR eligible for merge

Deploy trigger: Merge to main → PM2 reload (current) or Cloud Run deploy (production path)

3.5 Demo Seed Mode

When live scraping conditions are unstable (e.g., Cloudflare challenges, IP rate limiting),

DEMO_SEED=true activates fixture-based data injection from backend/tests/fixtures/. This ensures the pipeline can always complete a demonstrable end-to-end run without network dependencies.

Demo mode is controlled by an environment variable and never hard-coded.

3.6 Rollback Strategy

Severity	Trigger Condition	Action
P1 Critical	Search API 5xx for > 2 min OR GLM API auth failure	Immediate rollback to golden-release-v1 via PM2 / kubectl
P1 Critical	Database write failures on Listings table	Stop new sessions; restore from last DB backup
P2 High	Search latency P95 > 15s	Increase Uvicorn worker count; reduce MAX_LISTINGS_PER_SOURCE
P2 High	Scraper success < 70% on any source	Disable that scraper; continue with remaining sources
P3 Medium	Outreach message generation > 5s	Serve boilerplate fallback template immediately

4. Secure Integration & Credential Management

4.1 Credential Inventory

Homie integrates with three external services that require credentials. Each is managed separately to limit blast radius if any one secret is compromised.

Secret	Purpose	Storage	Rotation
GLM_API_KEY	ZhipuAI / ilmu.ai LLM calls	.env (server-side only)	On provider rotation or suspected leak
TELEGRAM_API_ID	Telethon MTProto session init	.env (server-side only)	Static — Telegram app credential
TELEGRAM_API_HASH	Telethon MTProto session init	.env (server-side only)	Static — Telegram app credential
TELEGRAM_PHONE	Telegram account phone number	.env (server-side only)	On account change
DATABASE_URL	PostgreSQL connection string	.env (server-side only)	On Neon branch rotation

4.2 Environment Variable Management

- All secrets are loaded exclusively from environment variables at runtime via Pydantic Settings (config.py). No secret is hard-coded in source.
- A .env file is used on the server for convenience. It is listed in .gitignore and never committed to the repository.
- The repository contains a .env.example file with placeholder values and documentation of each variable, committing actual values is a merge blocker.
- In CI, secrets are injected as masked environment variables through the CI provider's secret store (e.g., GitHub Actions Secrets), never embedded in workflow YAML.

4.3 Facebook Session Security

Facebook authentication is the most sensitive credential surface because it stores a long-lived browser session that grants account access.

- Cookies are obtained through a guided Playwright browser login flow triggered by the authenticated /api/facebook/login endpoint, not stored statically.
- The cookie file (fb_cookies.json) is stored server-side only, outside the repository, and is not served by any public endpoint.
- Session validity is checked on each scrape attempt via has_session() before any network call is made.
- If the session has expired, the pipeline degrades gracefully — Facebook results are skipped, and the user is notified to reconnect via /api/facebook/login.
- Planned: rotating session management to handle account-level rate limiting (identified in QATD as a future resilience improvement).

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.4 Telegram Credential Handling

- Telethon persists its MTProto session to telegram_session.session on disk after first authorisation. This file is excluded from version control via .gitignore.
- The session file grants Telegram account access equivalent to the phone number. It must be treated with the same sensitivity as a password.
- Automated outreach only initiates on explicit user confirmation, the system does not send messages autonomously.
- TELEGRAM_PHONE is stored in .env only. It is never logged or returned in any API response.

4.5 API Security Principles

- The backend does not expose any credential values through API responses. Endpoints return derived state (e.g., {"logged_in": true}) rather than raw secrets.
- Input validation via Pydantic rejects malformed requests (e.g., price_min > price_max returns HTTP 422) before any downstream processing occurs.
- The Facebook login endpoint (/api/facebook/login) should be protected behind authentication or network-level access control in any production deployment (not within hackathon scope).
- All scraping activity operates at human-browsing request rates with no persistent storage of full listing content beyond the session TTL (24 hours).
- CORS is configured to allow only the frontend origin. Wildcard origins are not permitted in production configuration.

4.6 Secret Hygiene Checklist

Control	Status
Secrets excluded from version control (.gitignore)	✓ Implemented
.env.example committed with placeholder values only	✓ Implemented
No secrets in API response bodies or logs	✓ Implemented
Facebook cookie file excluded from repository	✓ Implemented
Telegram session file excluded from repository	✓ Implemented
Credential rotation procedure documented	✓ This document